INF333 2025-2026 Spring Semester

Eray Aksoy <22401997@gsu.edu.tr>
Melissa Katz <22402112@gsu.edu.tr>

Homework I
Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

> We focused on passing the core tests and didn't implement any extra credit features

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

> We mostly relied on the Stanford Pintos documentation (PDF), the course slides, and some general StackOverflow threads to understand how fixed-point arithmetic works in C.

2 Sleep

2.1 Data Structures

Q3: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration.

Identify the purpose of each in 25 words or less.

`int64_t remaining_time_to_wake_up;` in `struct` thread: Keeps track of how many ticks are left before the thread wakes up.

`struct list_elem sleepelem;` in `struct` thread: Used to put the thread in the sleep list.

`struct list sleeping_list;` (Global in thread.c): A list holding all currently blocked/sleeping threads.

2.2 Algorithms

Q4: Briefly describe your implementation of `thread_join()` and how it interacts with thread termination.

We did not implement `thread_join()` and thread termination synchronization in this project since it is not part of Project 1. The tests for Project 1 focus primarily on alarm clock, priority scheduling, and MLFQS.

Q5: What steps are taken to minimize the amount of time spent in the timer interrupt handler?

In `thread_tick()`, we iterate through the `sleeping_list`. For each sleeping thread, we decrement its `remaining_time_to_wake_up`. If it reaches 0, we unblock it and remove it from the list. It takes O(N) time, but keeping a separate sleeping list is much better than iterating over all threads (including ready and blocked ones) in the system.

2.3 Synchronization

Q6: Consider parent thread `P` with child thread `C`. How do you ensure proper synchronization and avoid race conditions when `P` calls `wait(C)` before `C` exits? After `C` exits? How do you ensure that all resources are freed in each case? How about when `P` terminates without waiting, before `C` exits? After `C` exits? Are there any special cases?

As mentioned in Q4, we haven't implemented parent/child synchronization like `wait()` or `thread_join()` in this phase.

Q7: How are race conditions avoided when multiple threads call `timer_sleep()` simultaneously?

In `timer_sleep()`, we call `intr_disable()` before calling `thread_set_sleeping()`. This ensures that setting `remaining_time_to_wake_up`, adding the thread

to `sleeping_list`, and calling `thread_block()` happens atomically without any interruptions.

Q8: How are race conditions avoided when a timer interrupt occurs during a call to `timer_sleep()`?

Since interrupts are explicitly disabled using `intr_disable()` at the `beginning of timer_sleep()`, a timer interrupt cannot occur during the critical section. The interrupt will just pend until we re-enable interrupts.

2.4 Rationale

Q9: Critique your design, pointing out advantages and disadvantages in your design choices.

Our design uses a `sleeping_list` and decrements a counter (`remaining_time_to_wake_up`) for every sleeping thread on every tick. The advantage is that it is very intuitive and easy to debug. The disadvantage is performance: it modifies every sleeping thread's data on every tick (O(N)). A more advanced design would store the absolute wake-up tick and only check the minimum, but our approach was simpler to implement and passes all the alarm tests fine.

3 Priority Scheduling

3.1 Data Structures

Q10: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less

`int real_priority`: The thread's original priority before any donations.

`struct lock *current_lock`: The lock the thread is currently waiting on.

`struct list locks_held`: List of all locks the thread currently holds.

`int max_priority in struct lock`: The highest priority among threads waiting for this lock.

Q11: Describe the sequence of events when a call to `lock_acquire()` causes a priority donation. How is nested donation handled?

In `lock_acquire()`, if the lock has a holder, we set the thread's `current_lock`. Then we use a while loop to follow the `current_lock` and holder pointers upwards. If our priority is higher than the lock's `max_priority`, we update it, update the holder's priority, and rearrange the ready list if the holder is ready. This loop naturally handles nested donations by traversing the chain of locks.

Q12: Describe the sequence of events when `lock_release()` is called on a lock that a higher-priority thread is waiting for.

In `lock_release()`, we remove the lock from the thread's `locks_held list`. Then we call `thread_update_priority()`, which recalculates the thread's priority by checking its `real_priority` and the `max_priority` of all remaining locks in `locks_held`. Finally, we set lock->holder = NULL and call `sema_up()` to wake up the highest-priority waiter.

3.2 Synchronization

Q13: Describe a potential race in `thread_set_priority()` and explain how your implementation avoids it. Can you use a lock to avoid this race?

We avoid race conditions in `thread_set_priority()` by disabling interrupts (`intr_disable()`) while updating `real_priority` and calling `thread_update_priority()`. We cannot use a lock here because setting priority affects the fundamental scheduling behavior, and using locks could lead to circular dependencies or infinite loops since locks themselves rely on priority.

3.3 Rationale

Q14: Why did you choose this design? In what ways is it superior to another design you considered?

We chose to keep a `locks_held` list and dynamically recalculate the priority upon lock release. This is superior to trying to "undo" a specific donation because a thread might receive multiple donations from different locks. Recalculating from the `locks_held` list ensures we always revert to the correct priority, even in complex overlapping donation scenarios.

4 Advanced Scheduler

4.1 Data Structures

Q15: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

`int nice`: Thread's nice value.

`fixed_t recent_cpu`: Fixed-point representation of recent CPU usage.

`fixed_t load_avg` (Global): System-wide load average.

`FP_SHIFT_AMOUNT 15`: Defined in `fixed_point.h` for our 17.15 fixed-point arithmetic.

4.2 Algorithms

Q16: Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

timer	recent_cpu			priority			thread
ticks	A	B	C	A	B	C	to run
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A
24	16	8	0	59	59	59	C
28	16	8	4	59	59	58	A
32	20	8	4	58	59	58	B
36	20	12	4	58	58	58	C

Q17: Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

When priorities are equal, our scheduler relies on `try_thread_yield()` and the way `list_insert_ordered()` works. A yielding thread gets re-inserted, effectively placing it behind other threads of the same priority, which creates a round-robin behavior among ties. This resolved any ambiguity and matched the expected test behaviors.

Q18: How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

> Updating everything on every tick would be too slow. In our `thread_tick()`, we only increment `recent_cpu` for the running thread and update its priority every 4 ticks (`TIME_SLICE`). Heavy calculations (updating `load_avg` and `recent_cpu` for all threads) are done only once per second in `thread_tick_one_second()`. This division keeps the timer interrupt extremely fast.

4.3 Rationale

Q19: Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

> The design directly implements the formulas from the specification. The main advantage is that keeping the MLFQS math out of the main `thread_tick` path (except for simple increments) keeps the system responsive. If we had more time, we might optimize the `thread_foreach` loop in `thread_tick_one_second` to avoid checking inactive or dying threads.

Q20: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

> We decided to use C preprocessor macros defined in a new header file (`fixed_point.h`). We used a 15-bit fractional part (`FP_SHIFT_AMOUNT 15`). We used macros like `FP_ADD`, `FP_MULT_MIX` instead of functions to avoid the overhead of function calls inside the frequently executed timer interrupt. It made the code in `thread.c` much cleaner to read than having raw bit-shift operations everywhere.

5 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want–these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

> Overall it was not easy for sure but it's not something imposible thing to complete too. The priority donation logic, especially handling nested donations in `lock_acquire`, was the hardest part. Figuring out how to safely traverse the lock chain took a lot of time. Other parts were rather easy compared to this one.

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

> Yes, implementing the `MLFQS` math showed how much care is needed to balance CPU usage without floating-point units in a kernel.

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

> A hint about using a `locks_held` list for the priority release logic would have saved us some initial trial and error.

Q4: Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

> Maybe a short, practical session on how to effectively use GDB with Pintos would be great for the remaining projects, as debugging kernel panics was the most time-consuming part.

Q5: Any other comments?

> It was a tough but very instructive project.